Aetheris 项目审查与下一步规划

源码审查结论 · 生产化加固建议 · 后续实施路线

我的结论是：**Aetheris 已经形成了数据平台的主要功能框架，也有真实执行和持久化能力；但下一步最重要的不是继续扩充菜单，而是把权限、审批、执行和发布做成可靠的一整条链路。**

按大型企业的标准，我会把当前版本定位为“适合受控内部试用、仍需完成生产化加固”，暂不建议直接面向外部开放通用查询接口。

本次检查了路线图、关键后端链路、测试及 CI/CD 配置。以下是源码审查结论，不代表已经完成生产压测或运行态安全验收；本轮没有修改、提交或部署代码。

一、现在做到了什么程度

项目已经不是只有页面的演示系统：

- 数据源、SQL 查询、批量接入、任务执行和质量检查都有实际后端路径。
- 业务域、模型、维度、指标已经形成较清晰的语义资产体系。
- 草稿、发布快照、版本历史、审批和血缘都有实现基础。
- 数据集、看板、智能问数开始消费同一套语义能力。
- 已有后端测试、前端检查及跨服务冒烟测试。

这些基础值得保留，不需要推倒重写。

但目前存在一个突出问题：**单个模块里的控制措施，跨到另一种调用方式后，不一定仍然成立。**这比缺少几个功能更影响企业级可信度。

二、我认为必须优先解决的问题

1. 最高优先级：只读 SQL 守卫存在漏检

查询守卫使用 String.matches(".*关键词.*") 判断危险关键字，但没有处理跨行匹配。多行 SQL 中的禁止关键字可能漏过检查；普通查询连接也没有在该路径设置只读模式。

因此，“只允许查询”目前不能完全由这层代码保证。实际影响还取决于数据源账号的数据库权限，不能据此断言已经发生数据修改。

证据：JdbcService.java（第 521 行）。

建议采用多层防护：

- 按数据库方言解析并限制允许的 SQL 结构；
- 查询使用受限数据库账号；
- 在数据库支持时启用只读事务；
- 保留超时、行数、并发及危险函数限制。

验收重点不是“正常 SELECT 能运行”，而是多行、CTE、注释、嵌套及危险函数等反例都不能突破边界。

2. 高优先级：权限尚未覆盖所有执行入口

我发现了几个具体的不一致：

- 版本化指标查询没有传入具体指标 ID，细粒度授权无法准确匹配。
- 通用 SQL API 检查数据源权限，没有进一步约束 SQL 所访问的表和字段。
- 看板详情接口把看板 ID 当作数据集 ID 检查。
- 语义依赖图只检查登录身份，随后读取未按用户权限过滤的资产信息。

证据：版本化查询接口（第 27 行）；看板权限检查（第 110 行）；依赖图入口（第 11 行）。

这些问题既可能造成授权用户被误拒绝，也可能让部分路径突破预期的数据范围。

下一步应该统一传递“谁在调用、代表哪个应用、访问什么资源、允许哪些字段”的上下文，让控制台、数据集、Agent 和 API 在实际执行处遵守同一规则。内部服务令牌只能证明调用服务可信，不能替代最终用户的数据权限。

3. 高优先级：审批没有牢固绑定被审批的内容

目前批准记录按“资产 ID + 版本号”匹配，但修改草稿时保留原版本号。

这意味着存在这样的路径：提交内容 A → 审批通过 → 修改为内容 B → 仍以同一版本号通过发布检查。已有发布快照，并不等于审批时的内容也被冻结了。

证据：草稿更新保留版本号（第 129 行）；批准记录匹配方式（第 51 行）。

正确方向是：提交审批时保存不可变快照及内容摘要；修改内容后审批失效；发布只能发布批准过的快照。审批、发布状态切换还应有并发保护。

4. 高优先级：任务实例唯一，不等于任务只会执行一次

工作流抢占已有实例时，采用“先读状态、再更新”的方式。两个并发请求可能同时读到非运行状态，然后都开始执行。唯一索引能防止重复插入实例，不能防止同时更新并执行同一个已有实例。

证据：TaskExecutionService.java（第 96 行）。

这是需要并发测试验证的执行风险。接 Spark 前，应先落实原子抢占、执行租约、重启恢复，以及重试是否会重复写入的策略。

5. 审计已有入口，但还不能代表全部真实调用

当前语义查询审计放在部分公开控制器中，内部查询入口直接调用执行服务；通过 Serving、数据集等路径进入的调用，没有经过同一审计记录点。

另外，指标审计把 sourceTableId 传入了 datasourceId 参数，存在审计数据归属错误。

证据：指标审计调用（第 55 行）；内部指标查询（第 21 行）。

应把审计下沉到统一执行边界，记录用户、应用、资源版本、调用链 ID、权限决策、结果和耗时。否则页面上的成功率、P95 只代表被记录的那部分流量。

6. 测试和发布流程还没有形成完整门禁

项目已有 CI 和冒烟测试，这是好的基础。但新增 Serving 安全入口缺少对应的 HTTP 回归；已有控制器测试仍直接实例化旧兼容构造器，没有验证真实 HTTP 身份与权限链路。

部署方面也存在明确不一致：

- metadata、quality、governance 在 Compose 中只有构建配置，没有镜像地址；
- 部署脚本主要执行拉镜像和启动，不能保证这三个服务更新；
- 镜像发布工作流独立响应 push，没有落实注释中“CI 通过后发布”的依赖；
- 部署没有把健康等待和业务冒烟作为成功条件。

证据：控制器测试（第 33 行）；Compose（第 43 行）；镜像发布流程；部署流程。

三、下一步应该怎么排

我建议调整路线图执行顺序，而不是继续机械地勾选“已完成”。

顺序	核心交付	验收标准
第一轮： 安全与执行加固	SQL 守卫、统一权限上下文、审批快照、任务原子抢占	越权请求在各入口均被拒绝；审批后修改不能直接发布；并发触发只有一个执行者
第二轮： 治理闭环与可靠发布	全入口审计、跨资产依赖、下线阻断、固定版本镜像、迁移与恢复演练	能说明资产被谁使用、下线影响谁；同一版本可重复部署，并有经过验证的恢复方案
第三轮： 真正的数据服务开放	应用注册、服务资产绑定、不可变发布版本、密钥轮换吊销、持久化配额和调用监控	外部应用只能调用被授权的具体服务版本；吊销及时生效，调用可追溯
后续业务扩展	Spark 作业、多表语义、派生指标等	每项能力都有真实执行、故障处理及端到端测试，不只新增配置页面

第三轮需要特别区分：**URL 带 /v1 是接口协议版本，不等于已经完成“数据服务版本管理”。**后者还需要冻结查询定义、参数契约、资源绑定及发布记录。

对外开放时，我更建议先提供参数化、已注册的指标或查询服务，而不是优先开放任意 SQL。

四、Spark 应该什么时候做

Spark 不必等 Kubernetes、服务市场和所有治理功能全部完成。

目前已有 Spark/Hive JDBC 路径，但这与“提交和管理独立 Spark 作业”是两种能力。完成第一轮执行安全加固后，就可以把 Spark 作为一个明确的后续交付，先做一条完整路径：

提交作业 → 保存真实作业 ID → 查询状态和日志 → 取消 → 失败重试 → 验证结果。

接入前先统一执行器的状态、超时、取消、幂等和资源身份协议，避免 Spark 再形成一套独立的任务体系。

我的最终建议

下一轮就做“生产化基础加固”，暂缓新增菜单和对外开放范围。

选一条真实业务链路作为验收主线：数据接入、任务加工、质量检查、指标审批、看板消费和 API 查询。用管理员、受限用户、未授权用户分别走通正常与失败场景，并验证重试、撤权、下线和部署恢复。

Aetheris 当前最需要的，是把已经做出来的功能变成可以放心依赖的能力。做到这一点以后，再接 Spark、扩展语义计算和开放数据服务，后续开发才不会反复返工。